FACULDADE FACINT

ENGENHARIA DE SOFTWARE

Fundamentos de Estruturas de Dados

Organização e Manipulação de Estruturas de Dados

Dados · Organização · Operações · Eficiência · Big O

🎯 Objetivos de aprendizagem

🧩 Compreender
o conceito de estrutura de dados e a diferença entre dado e informação

🗂️ Diferenciar
tipos de dados e estruturas estáticas × dinâmicas

⚙️ Aplicar
operações básicas: inserir, remover, acessar, buscar e percorrer

⏱️ Analisar
a eficiência de algoritmos em tempo e espaço

📈 Interpretar
a notação Big O em casos simples

💻 Relacionar
estruturas de dados a sistemas reais

🧱 O que são estruturas de dados?

Uma **estrutura de dados** é uma forma de **organizar, armazenar e acessar** dados na memória para que possam ser usados **de maneira eficiente**.

- Define **como** os dados ficam dispostos
- Define **quais operações** são possíveis
- Influencia o **desempenho** do programa

Analogia

Uma biblioteca organiza livros por estante, seção e código. Sem organização, encontrar um livro exigiria olhar **todos** eles.

Estrutura de dados = **dados** + **organização** + **operações**

🔎 Dados × Informação

Dado bruto
7.5, 8.0, 6.0

➜

Organização
estrutura + contexto

➜

Processamento
média, ordenação

➜

Informação
"Ana foi aprovada"

Dado

Valor isolado, sem contexto: `8.0`, `"Ana"`, `true`

Informação

Dado **organizado e interpretado** que apoia uma decisão

💡 A estrutura de dados é a **ponte** entre o dado bruto e a informação útil.

🗂️ Por que organizar os dados?

❌ Sem organização

- Buscas lentas
- Dados duplicados
- Código difícil de manter
- Desperdício de memória

✅ Com organização

- Acesso **rápido**
- Operações **previsíveis**
- Código **legível**
- Uso **racional** de recursos

> **Representação na memória:** um array guarda elementos em posições **contíguas**, identificadas por índices.

`"Ana"`	`"Carlos"`	`"Mariana"`	`"João"`
`[0]`	`[1]`	`[2]`	`[3]`

💻 Estruturas de dados no software

🛒 E-commerce
Carrinho de compras como **lista**

↩️ Editores
Desfazer (Ctrl+Z) com **pilha**

🖨️ Impressão
Fila de documentos com **fila**

🗺️ Mapas
Rotas e caminhos com **grafos**

📁 Sistemas de arquivos
Pastas com **árvores**

🔐 Login
Sessões com **tabela hash**

🗄️ Bancos de dados
Índices com **árvores B**

🌐 Navegador
Histórico com **pilha/lista**

Escolher a estrutura certa é uma **decisão de engenharia** que afeta desempenho, custo e escalabilidade.

Tipos de dados

Primitivos (valor único)

Tipo	Exemplo
`number`	`42`, `3.14`
`string`	`"Ana"`
`boolean`	`true`
`null` / `undefined`	ausência de valor

Compostos (agrupam valores)

Tipo	Exemplo
`Array`	`[1, 2, 3]`
`Object`	`{ nome: "Ana" }`
`Map` / `Set`	pares e conjuntos

> Tipos **abstratos de dados (TAD)** — como pilha e fila — definem **o comportamento**; a estrutura define **como implementá-lo**.

🚀 Primeiro exemplo em JavaScript

```
// Criamos um array para armazenar os nomes dos alunos
const alunos = ["Ana", "Carlos", "Mariana"];

// Adicionamos um novo elemento ao final da estrutura
alunos.push("João");

// Acessamos diretamente uma posição pelo índice (começa em 0)
console.log(alunos[0]);      // "Ana"
console.log(alunos[3]);      // "João"

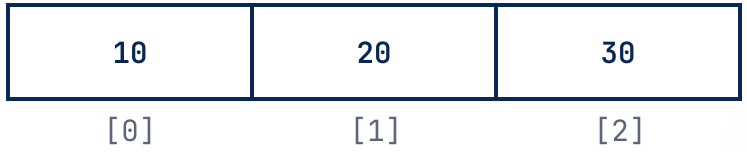
// A propriedade length informa o tamanho atual da estrutura
console.log(alunos.length);  // 4
```

📌 **O que demonstra:** o array é uma estrutura **linear e indexada**. Cada elemento ocupa uma posição numerada, o que permite **acesso direto** e **inserção no final**.

⚖️ Estruturas estáticas e dinâmicas

📏 Estáticas

- Tamanho **definido** na criação
- Memória **contígua**
- Acesso por índice muito rápido
- Ex.: vetores em C, `Int32Array` em JS



🔗 Dinâmicas

- Tamanho **varia** em execução
- Memória pode ser **não contígua**
- Crescem e diminuem sob demanda
- Ex.: listas encadeadas, `Array` em JS



🛠️ Operações básicas

➕ Inserção

Adicionar um elemento (início, meio ou fim)

➖ Remoção

Retirar um elemento da estrutura

🎯 Acesso

Ler o elemento de uma posição específica

🔍 Busca

Localizar um elemento pelo seu valor

🔁 Percurso

Visitar todos os elementos, um a um

✏️ Atualização

Alterar o valor de um elemento existente

Cada estrutura executa essas operações com **custos diferentes** — e é isso que guia a escolha.

🧪 Exemplo prático: operações

```
const fila = ["Ana", "Carlos", "Mariana"];

fila.push("João");            // Inserção no FIM     → [Ana, Carlos, Mariana, João]
fila.unshift("Bia");          // Inserção no INÍCIO  → desloca todos os elementos
fila.shift();                 // Remoção do INÍCIO   → remove "Bia"
fila.splice(1, 1);            // Remoção na posição 1 → remove "Carlos"

fila[0] = "Ana Paula";        // Atualização por índice

const pos = fila.indexOf("João");      // Busca: retorna a posição ou -1
console.log(pos);                      // 2

for (const nome of fila) {             // Percurso: visita cada elemento
  console.log(nome);
}
```

📌 **Conceito:** todas as operações básicas em uma única estrutura. Note que inserir/remover no **início** obriga o array a **deslocar** os demais elementos — isso tem custo!

⚡ Eficiência de algoritmos

Dois programas podem dar o **mesmo resultado**, mas com **custos muito diferentes**.

- Quantos **passos** o algoritmo executa?
- Como esse número **cresce** quando os dados aumentam?
- Funciona bem com **10** itens... e com **10 milhões**?

Situação real

Uma busca que leva 1 ms com 100 clientes pode levar **minutos** com 10 milhões se o algoritmo for ineficiente.



⏱️ Tempo e 💾 Espaço

⏱️ Complexidade de tempo

Quantidade de **operações** executadas em função do tamanho da entrada.

```
// Percorre n elementos → n passos
let soma = 0;
for (const x of numeros) soma += x;
```

💾 Complexidade de espaço

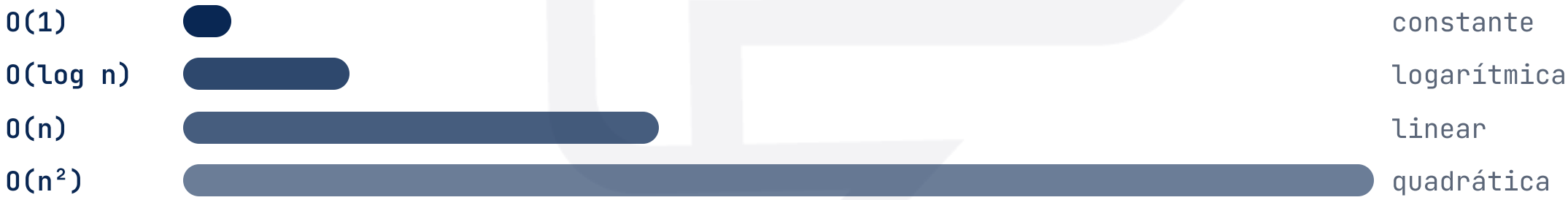
Quantidade de **memória extra** utilizada pelo algoritmo.

```
// Cria uma cópia → n posições extras
const copia = [...numeros];
```

⚖️ Muitas vezes existe um **trade-off**: gastar mais memória para ganhar velocidade, ou o contrário.

📈 Introdução à notação Big O

Big O descreve como o custo de um algoritmo **cresce** no **pior caso**, ignorando constantes.



Operação em array	Big O
Acessar `arr[i]` / `push`	**O(1)**
Buscar com `indexOf` / `unshift`	**O(n)**
Comparar todos com todos	**O(n²)**

🧭 Big O no dia a dia: analogias

🔑 O(1) · Constante

Abrir o **armário de número 42**: você vai direto nele, existam 50 ou 5.000 armários.

JS: `arr[42]`

📖 O(log n) · Logarítmica

Procurar uma palavra no **dicionário**: abre no meio e descarta metade a cada passo.

JS: busca binária em array ordenado

🎟️ O(n) · Linear

Conferir o **ingresso de cada pessoa** da fila: dobrou a fila, dobrou o tempo.

JS: `indexOf`, `for...of`

🤝 O(n²) · Quadrática

Numa festa, **todos cumprimentam todos**: 10 pessoas → ~45 apertos; 1.000 → ~500 mil.

JS: laço dentro de laço

💡 Pergunta-chave: **se os dados dobrarem, o que acontece com o tempo?** Nada (O(1)) · +1 passo (O(log n)) · dobra (O(n)) · quadruplica ($O(n^2)$).

🧠 Complexidade na prática

```
const ids = [12, 45, 7, 89, 33];

// O(1) → acesso direto: 1 passo, não importa o tamanho do array
const primeiro = ids[0];

// O(n) → busca linear: no pior caso visita TODOS os n elementos
function buscar(lista, alvo) {
  for (let i = 0; i < lista.length; i++)
    if (lista[i] === alvo) return i;   // encontrou: retorna a posição
  return -1;                           // não encontrou
}

// O(n²) → laço dentro de laço: compara cada par de elementos
function temDuplicado(lista) {
  for (let i = 0; i < lista.length; i++)
    for (let j = i + 1; j < lista.length; j++)
      if (lista[i] === lista[j]) return true;
  return false;
}
```

📌 **Conceito:** a mesma estrutura, com operações de custos **O(1)**, **O(n)** e **O(n²)**.

🔄 Comparando soluções

```
const lista = [12, 45, 7, 89, 33, 45];

// Solução 1 — O(n²): dois laços aninhados (slide anterior)
temDuplicado(lista);                    // true

// Solução 2 — O(n): um Set guarda o que já foi visto
function temDuplicadoRapido(lista) {
  const vistos = new Set();             // estrutura com busca O(1) em média
  for (const x of lista) {
    if (vistos.has(x)) return true;     // já apareceu antes → duplicado
    vistos.add(x);                      // registra o elemento
  }
  return false;
}
```

⏱️ **Tempo:** de O(n²) para **O(n)**

💾 **Espaço:** de O(1) para **O(n)** — o trade-off!

🌍 Aplicação em sistemas reais

🔎 Busca de produtos

Índices e hash evitam percorrer milhões de registros a cada consulta

📱 Feed de redes sociais

Filas de prioridade ordenam o que aparece primeiro

🚗 Apps de rota

Grafos e algoritmos de caminho mínimo calculam trajetos

Requisito → **Operações mais frequentes** → **Análise Big O** → **Estrutura escolhida**

Síntese dos conceitos

- **Estrutura de dados** = dados + organização + operações
- **Dado** vira **informação** quando organizado e interpretado
- Tipos **primitivos** e **compostos**
- Estruturas **estáticas** × **dinâmicas**
- Operações: **inserir, remover, acessar, buscar, percorrer**
- Eficiência medida em **tempo** e **espaço**
- **Big O** descreve o crescimento do custo
- A escolha da estrutura **impacta o software**

🧭 Não existe estrutura "melhor": existe a estrutura **mais adequada ao problema**.

📝 Reflexão

```
// Desafio: complete as funções e informe o Big O de cada uma
const notas = [7.5, 9.0, 6.0, 8.5, 5.0];

function maiorNota(lista) { /* percorra e retorne a maior */ }
function contarAprovados(lista, media) { /* conte ⩾ media */ }
function removerPrimeira(lista) { /* qual o custo de shift()? */ }
```

💬 Reflita

1. Que estrutura você usaria para um **histórico de navegação**? Por quê?
2. Em um sistema com **milhões de usuários**, por que $O(n^2)$ é um risco?
3. Cite uma situação em que vale **gastar memória** para ganhar tempo.